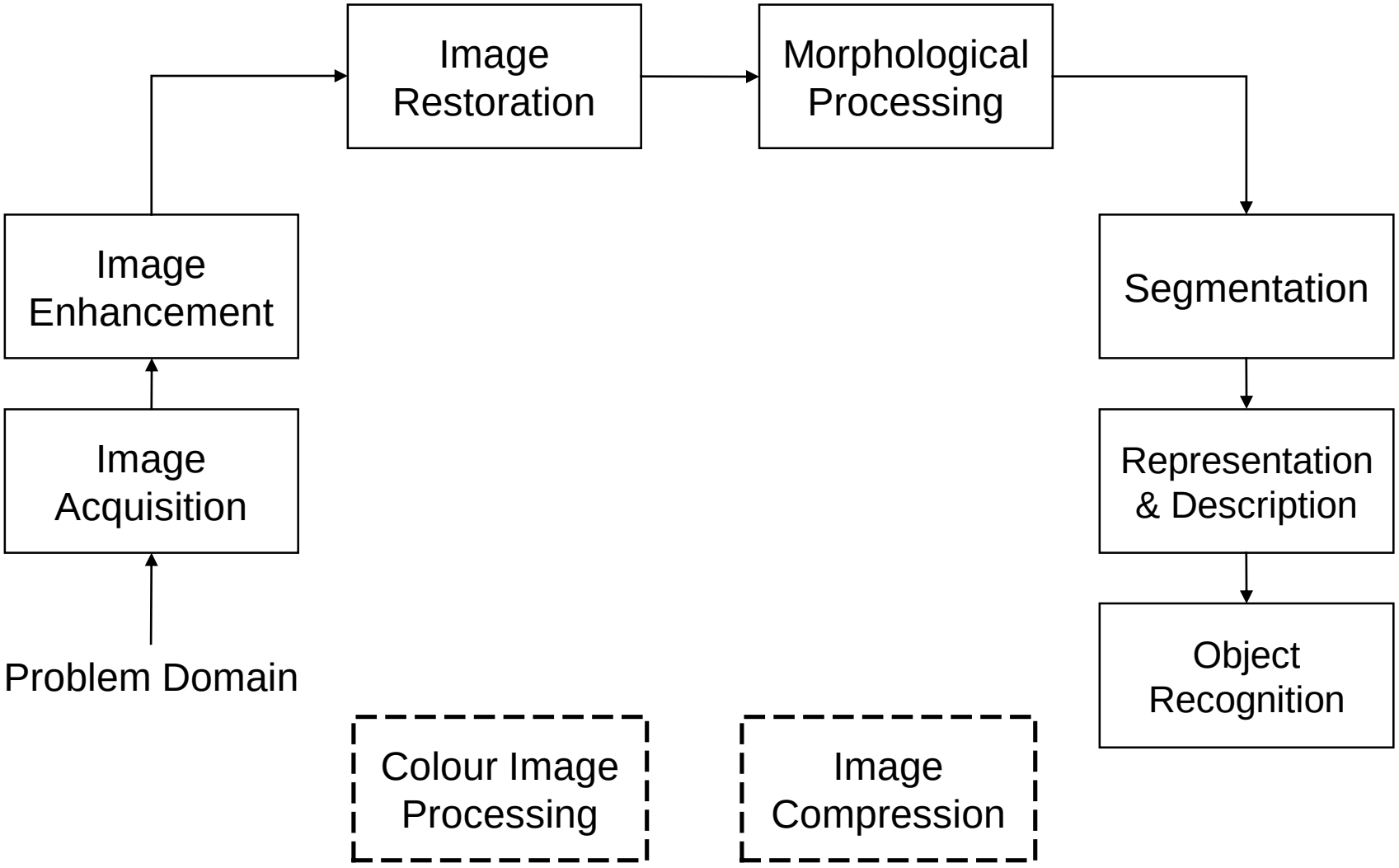
Image Compression

Content derived from multiple sources

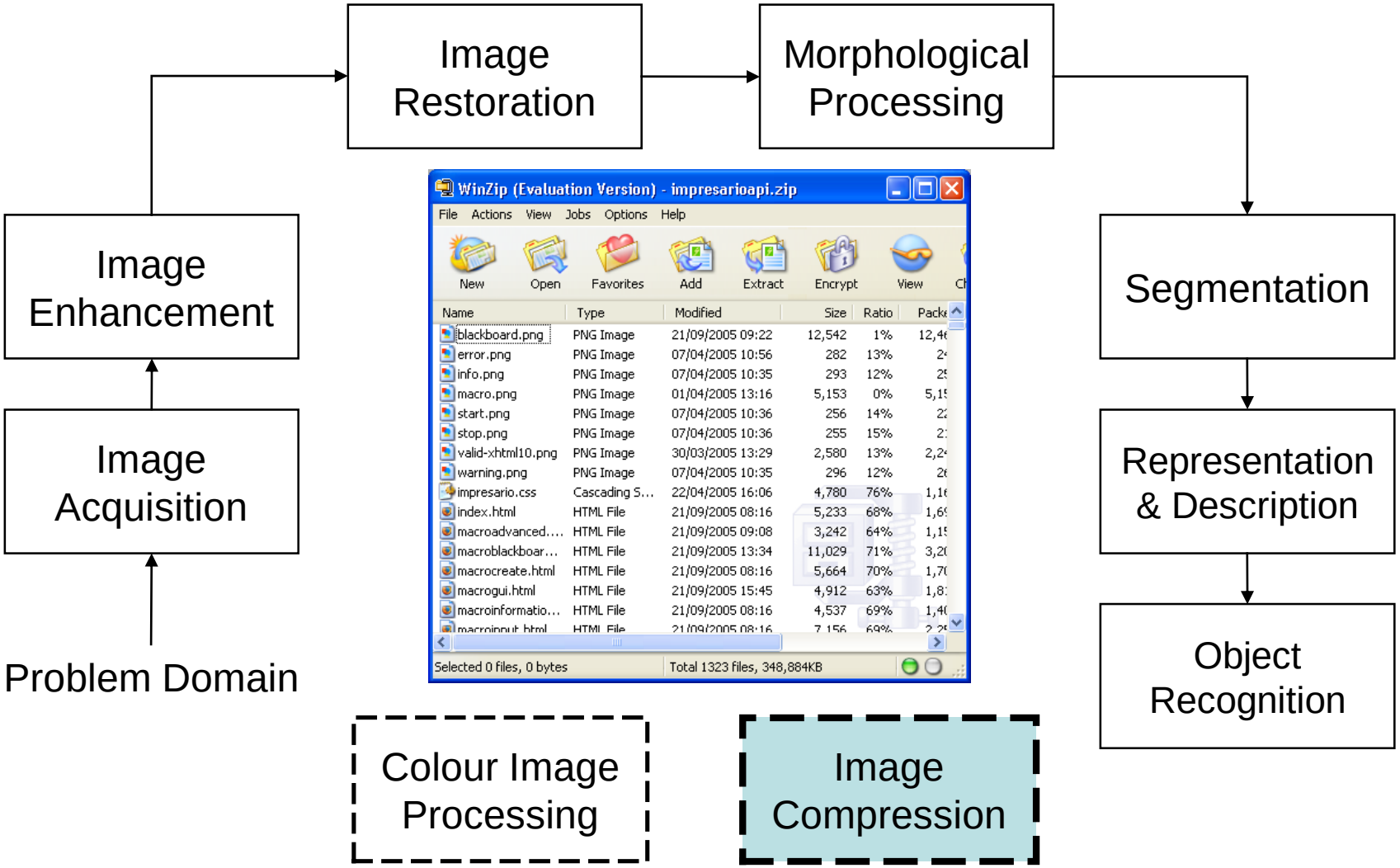
Contents

- ✓ Overview
- ✓ Lossless Compression Techniques
  - Run Length Coding
  - Shannon-Fano Coding
  - Huffman Coding
  - Arithmetic Coding
- ✓ Lossy Compression Techniques
  - Graylevel Run Length Coding
  - Block Truncation Compression
  - Vector Quantization Compression
  - Transform Coding

Phases of Digital Image Processing



Phases of Digital Image Processing : Image Compression



Introduction and Overview

The field of image compression continues to grow at a rapid pace.

In future, the need to store and transmit images will only continue to increase faster than the available capability to process all the data.

Purpose of Compression

- Gray Scale 8-bit Image: 720×480 = 345.6 KB
- Color Image: 720×480 = 1.04 MB
- 90 minutes Color Video: 720×480×30frames/sec=168.48 GB
- Internet Speed: 56 Kbps
- Approx. download time: 51 seconds, 156 seconds and 299 days, respectively.
- Outcomes of Compression:
  - ✓ Reduce Storage Space
  - ✓ Reduce Transmission Time
  - ✓ Reduce Data access Time

Different Applications of Image Compression

- Internet
- Satellite imaging
- Medical imaging
- Multimedia
- Businesses

Compression algorithm development starts with applications to two-dimensional (2-D) still images.

After the 2-D methods are developed, they are often extended to video (motion imaging).

We will discuss image compression of single frames of image data.

Image compression involves reducing the size of image data files, while retaining necessary information.

Retaining necessary information depends upon the application.

Image segmentation methods, which are primarily a data reduction process, can be used for compression.

The reduced file created by the compression process is called the compressed file and is used to reconstruct the image, resulting in the decompressed image.

The original image, before any compression is performed, is called the uncompressed image file.

The ratio of the uncompressed image file and the compressed file is referred to as the Compression Ratio.

Compression Ratio

$$Compression\ Ratio = \frac{Uncompressed\ file\ size}{Compressed\ file\ size} = \frac{SIZE_U}{SIZE_C}$$

$$Often\ writen\ as\ \rightarrow\ SIZE_U : SIZE_C$$

EXAMPLE 10.1.1: The original image is 256x256 pixels, single-band (grayscale), 8-bits per pixel. This file is 65,536 bytes (64k). After compression the image file is 6,554 bytes. The compression ratio is: $SIZE_U/SIZE_C = 65536/6554 = 9.999 \approx 10$. This can also be written as 10:1

This is called a "10 to 1 compression", a "10 times compression", or can be stated as

"compressing the image to 1/10 its original size". Another way to state the compression is to use

the terminology of *bits per pixel*. For an NxN image:

$$Bits\ per\ pixel = \frac{Number\ of\ bits}{Number\ of\ pixels} = \frac{(8)(Number\ of\ bytes)}{NxN}$$

EXAMPLE 10.1.2: Using the preceding example, with a compression ratio of 65,536/6,554 bytes, we want to express this as bits per pixel. This is done by first finding the number of pixels in the image: 256x256 = 65,536 pixels. We then find the number of bits in the compressed image file: (6,554 bytes)(8 bits/byte) = 52,432 bits. Now we can find the bits per pixel by taking the ratio: 52,432/65,536 = 0.8 bits/pixel

The reduction in file size is necessary to meet the bandwidth requirements for many transmission systems, and for the storage requirements in computer databases.

Types of Image Compression Methods

1. Lossless Compression Methods:

- Allows for the exact restoration of the original image data.

- It can compress complex images to a maximum 1/2 to 1/3 the original size – 2:1 to 3:1 compression ratios.

- Preserves the data exactly.

2. Lossy Compression Methods:

- Data loss, original image cannot be re-created exactly.

- It can compress complex images 10:1 to 50:1 and retain high quality, and 100 to 200 times for lower quality, but acceptable images.

Compression algorithms are developed by taking advantage of the redundancy that is inherent in image data.

Four primary types of redundancy that can be found in images are:

1. Coding
2. Interpixel
3. Interband / Chromatic
4. Psychovisual redundancy

1. Coding Redundancy

It occurs when the data used to represent the image is not utilized in an optimal manner.

2. Interpixel Redundancy

It occurs because adjacent pixels tend to be highly correlated, in most images the brightness levels do not change rapidly, but change gradually.

3. Interband /Chromatic Redundancy

Occurs in color images due to the correlation between bands within an image – if we extract the red, green, and blue bands they look similar.

4. Psychovisual Redundancy

Some information is more important to the human visual system than other types of information.

The key in image compression algorithm development is to determine the minimal data required to retain the necessary information.

The compression is achieved by taking advantage of the redundancy that exists in images.

If the redundancies are removed prior to compression, for example with a decorrelation process, a more effective compression can be achieved.

Compression System Model

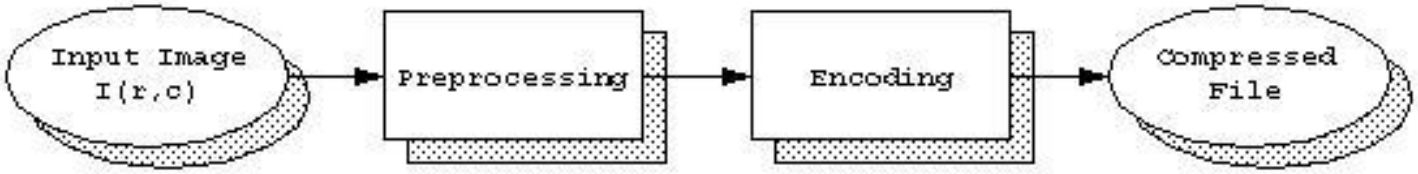
It consists of two parts:

1. The compressor
2. The decompressor

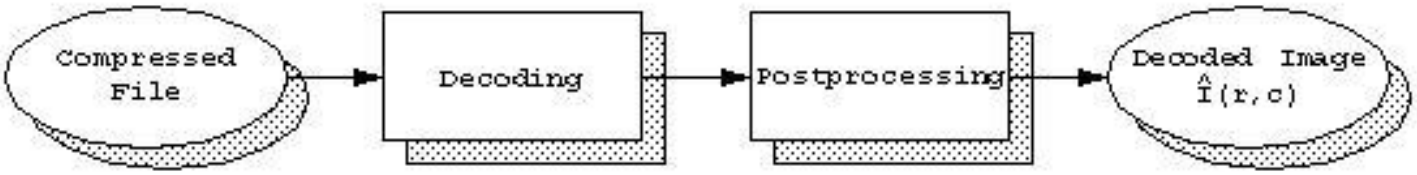
The compressor consists of a preprocessing stage and encoding stage.

The decompressor consists of a decoding stage followed by a postprocessing stage.

Figure 10.1-1: Compression System Model



a) Compression



b) Decompression

Before encoding, preprocessing is performed to prepare the image for the encoding process, and consists of any number of operations that are application specific.

After the compressed file has been decoded, postprocessing can be performed to eliminate some of the potentially undesirable artifacts brought about by the compression process.

The development of a compression algorithm is highly application specific.

Preprocessing stage of compression consists of processes such as enhancement, noise removal, or quantization.

The goal of preprocessing is to prepare the image for the encoding process by eliminating any irrelevant information, where irrelevant is defined by the application.

For example, many images that are for viewing purposes only can be preprocessed by eliminating the lower bit planes, without losing any useful information.



a) Original image



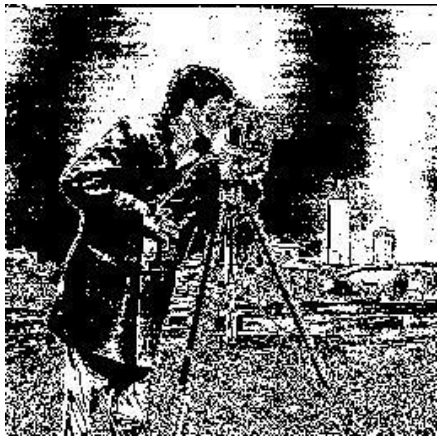
b) Bit plane 7, the most significant bit



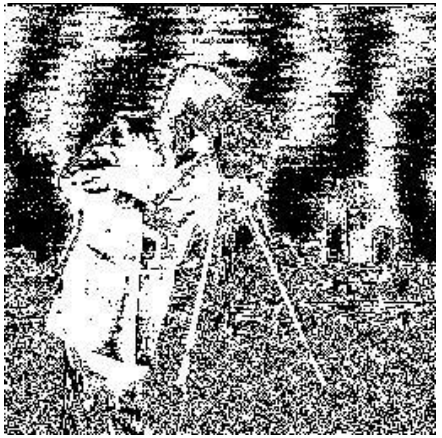
c) Bit plane 6



d) Bit plane 5



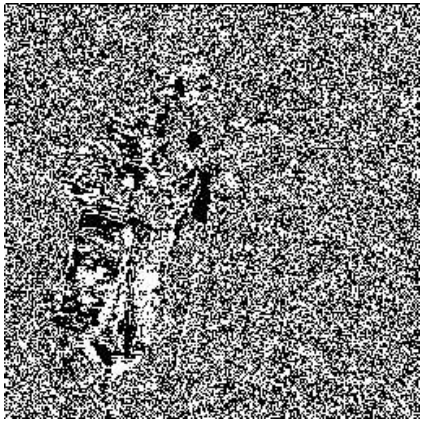
e) Bit plane 4



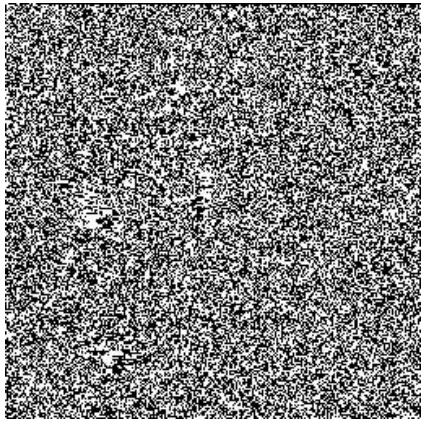
f) Bit plane 3



g) Bit plane 2



h) Bit plane 1



i) Bit plane 0,
the least significant bit

Lossless Compression Methods

No loss of data, decompressed image exactly same as uncompressed image.

Medical images or any images used in benches.

Lossless compression methods typically provide about a 10% reduction in file size for complex images.

The underlying theory for lossless compression (also called *data compaction*) comes from the area of communications and information theory, with a mathematical basis in probability theory.

One of the most important concepts used is the idea of information content and randomness in data.

Entropy is the measurement of the average information content in an image.

The entropy for an $N \times N$ image is calculated by this equation:

$$Entropy = -\sum_{i=0}^{L-1} p_i \log_2 p_i \text{ (in bits/pixel)}$$

Where,

p_i= the probability of the $i^{th} gray\ level = \frac{n_i}{N^2}$

n_i= the total number of pixels with gray value i.

L= the total number of gray levels.

This measure provides us with a theoretical minimum for the average number of bits per pixel that could be used to code the image.

It can also be used as a metric for judging the success of a coding scheme, as it is theoretically optimal.

EXAMPLE 10.2.1:

Let L = 8, meaning there are 3 bits/pixel in the original image. Now, let's say the number of pixels at each gray level value is equal (they have the same probability), that is:

$$p_0 = p_1 = \ldots = p_7 = \frac{1}{8}$$

Now, we can calculate the entropy as follows:

$$Entropy = -\sum_{i=0}^{7} p_i \log_2(p_i) = -\sum_{i=0}^{7} \frac{1}{8} \log_2\left(\frac{1}{8}\right) = 3$$

This tells us that the theoretical minimum for lossless coding for this image is 3 bits per pixel. In other words, there is no code that will provide better results than the one currently used (called the natural code, since $000_2 = 0, 001_2 = 1, 010_2 = 2, \ldots, 111_2 = 7$). This example illustrates that the image with the most random distribution of gray levels, a uniform distribution, has the highest entropy

The range of the entropy:

$$0 \leq Entropy \leq \log_2(L)$$

The examples also illustrate the information theory perspective regarding information and randomness.

The more randomness that exists in an image, the more evenly distributed the gray levels, and more bits per pixel are required to represent the data.



a) Original image,
entropy = 7.032 *bpp*



b) Image after local histogram equalization,
block size 4, entropy = 4.348 *bpp*



c) Image after binary threshold,
entropy = 0.976 *bpp*

Run-Length Coding

Run-Length Coding

Run-length coding (RLC) works by counting adjacent pixels with the same gray level value called the *run-length,* which is then encoded and stored.

RLC works best for binary, two-valued, images.

RLC can also work with complex images that have been preprocessed by thresholding to reduce the number of gray levels to two.

RLC can be implemented in various ways, but the first step is to define the required parameters.

Horizontal RLC (counting along the rows) or vertical RLC (counting along the columns) can be used.

In basic horizontal RLC, the number of bits used for the encoding depends on the number of pixels in a row.

If the row has 2^n pixels, then the required number of bits is *n*, so that a run that is the length of the entire row can be encoded.

EXAMPLE 10.2.5:

A 256x256 image requires 8-bits, since $2^8 = 256$.

EXAMPLE 10.2.6:

A 512x512 image requires 9-bits, since $2^9 = 512$.

The next step is to define a convention for the first RLC number in a row – does it represent a run of 0's or 1's?

EXAMPLE 10.2.7:

The image is an 8x8 binary image, which requires 3 bits for each run-length coded word. In the actual image file are stored 1's and 0's, although upon display the 1's become 255 (white) and the 0's are 0 (black). To apply RLC to this image, using horizontal RLC:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

EXAMPLE 10.2.7 (contd):

The RLC numbers are:

First row: 8

Second row: 0, 4, 4

Third row: 1, 2, 5

Fourth row: 1, 5, 2

Fifth row: 1, 3, 2, 1, 1

Sixth row: 2, 1, 2, 2, 1

Seventh row: 0, 4, 1, 1, 2

Eighth row: 8

Note that in the second and seventh rows, the first RLC number is 0, since we are using the convention that the first number corresponds to the number of zeros in a run

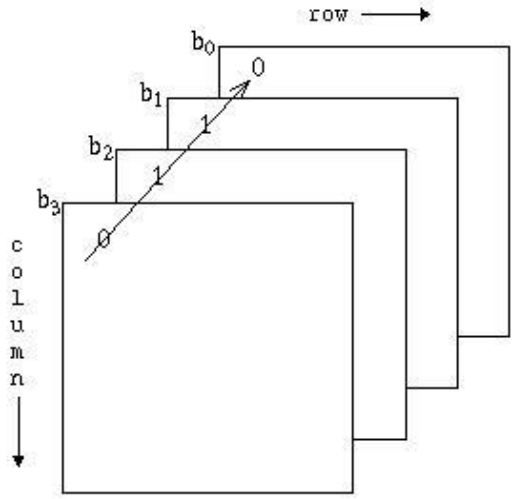
Bitplane-RLC : A technique which involves extension of basic RLC method to gray level images, by applying basic RLC to each bit-plane independently.

For each binary digit in the gray level value, an image plane is created, and this image plane (a string of 0's and 1's) is then encoded using RLC.

Figure 10.2-4: Bit Plane Run-length Coding

b_3	b_2	b_1	b_0
0	0	0	0
0	0	0	1
0	0	1	0
.	.	.	.
.	.	.	.
.	.	.	.
0	1	1	0
1	1	1	1

a) 4 bits/pixel designation



b) bit-planes → b3 b2 b1 b0
0 1 1 0

Another way to extend basic RLC to gray level images is to include the gray level of a particular run as part of the code.

Here, instead of a single value for a run, two parameters are used to characterize the run.

The pair (G,L) correspond to the gray level value G, and the run length L.

This technique is only effective with images containing a small number of gray levels.

EXAMPLE 10.2.9:

Given the following 8x8, 4-bit image:

$$\begin{bmatrix} 10 & 10 & 10 & 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 & 12 & 12 & 12 \\ 10 & 10 & 10 & 10 & 10 & 12 & 12 & 12 \\ 0 & 0 & 0 & 10 & 10 & 10 & 0 & 0 \\ 5 & 5 & 5 & 0 & 0 & 0 & 0 & 0 \\ 5 & 5 & 5 & 10 & 10 & 9 & 9 & 10 \\ 5 & 5 & 5 & 4 & 4 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

EXAMPLE 10.2.9 (contd):

The corresponding gray levels pairs are as follows:

First row: 10,8

Second row: 10,5 12,3

Third row: 10,5 12,3

Fourth row: 0,3 10,3 0,2

Fifth row: 5,3 0,5

Sixth row: 5,3 10,2 9,2 10,1

Seventh row: 5,3 4,3 0,2

Eighth row: 0,8

These numbers are then stored in the RLC compressed file as:

10,8,10,5,12,3,10,5,12,3,0,3,10,3,0,2,5,3,0,5,5,3,10,2,9,2,10,1,5,3,4,3,0,2,0,8

The decompression process requires the number of pixels in a row, and the type of encoding used.

Standards for RLC have been defined by the International Telecommunications Union-Radio (ITU-R, previously CCIR).

These standards use horizontal RLC, but postprocess the resulting RLC with a Huffman encoding scheme.

Newer versions of this standard also utilize a two-dimensional technique where the current line is encoded based on a previous line, which helps to reduce the file size.

These encoding methods provide compression ratios of about 15 to 20 for typical documents.

Uniquely Decodable Codes

Variable length code assigns a bits of string (codeword) of variable length to every message value

e.g. a = 1, b = 01, c = 101, d = 011

What if you get the sequence of bits 1011 ?

Is it aba, ca, or, ad?

Uniquely decodable code is a variable length code in which bit strings can always be uniquely decomposed into its codewords.

Prefix Codes

Prefix code is a variable length code in which no codeword is a prefix of another word.

It is possible to decode each message in an encoded string without reference to succeeding code symbols.

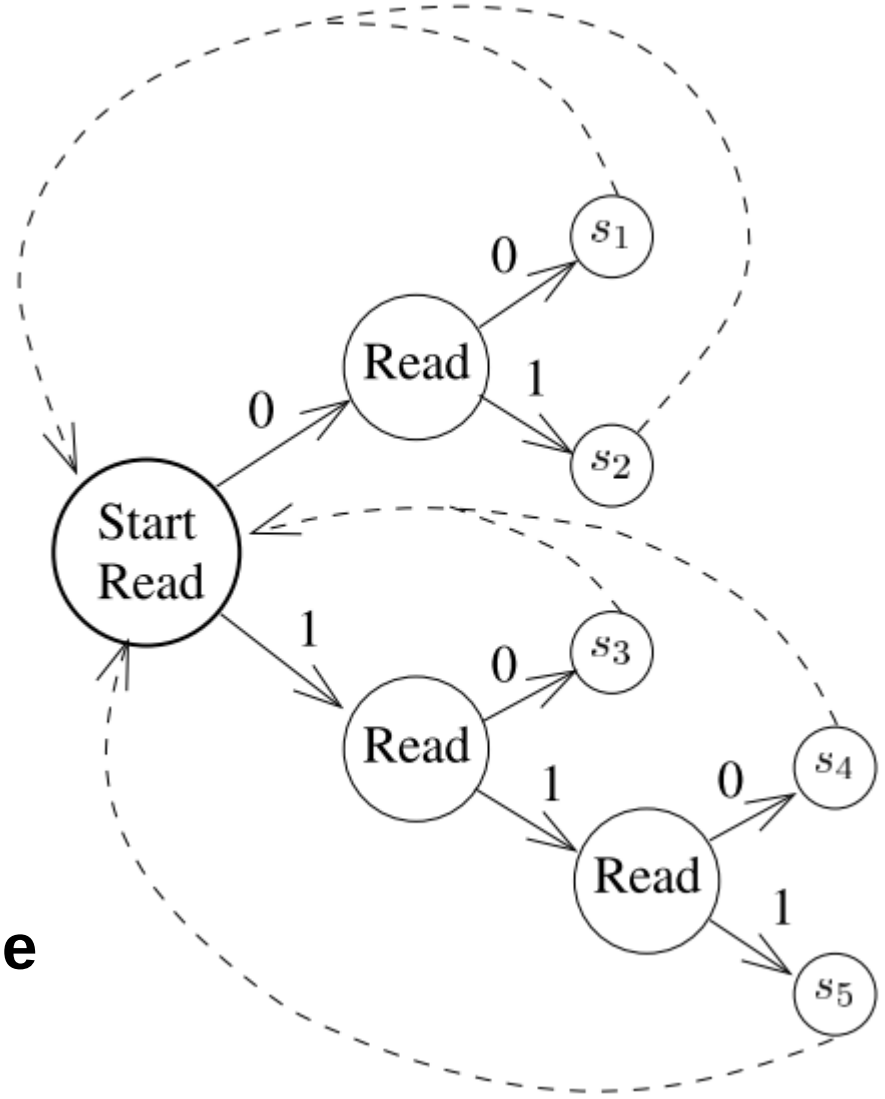
Example

Source	Code 1	Code 2
a_1	0	0
a_2	10	01
a_3	110	011
a_4	1110	0111

Code1 is Prefix code.

Decoding Prefix Codes

Source	Code
s_1	00
s_2	01
s_3	10
s_4	110
s_5	111



An example of Prefix code is Shannon Fano code or Huffman code.

Shannon Fano Coding

[Claude Shannon & Robert Fano, 1949]

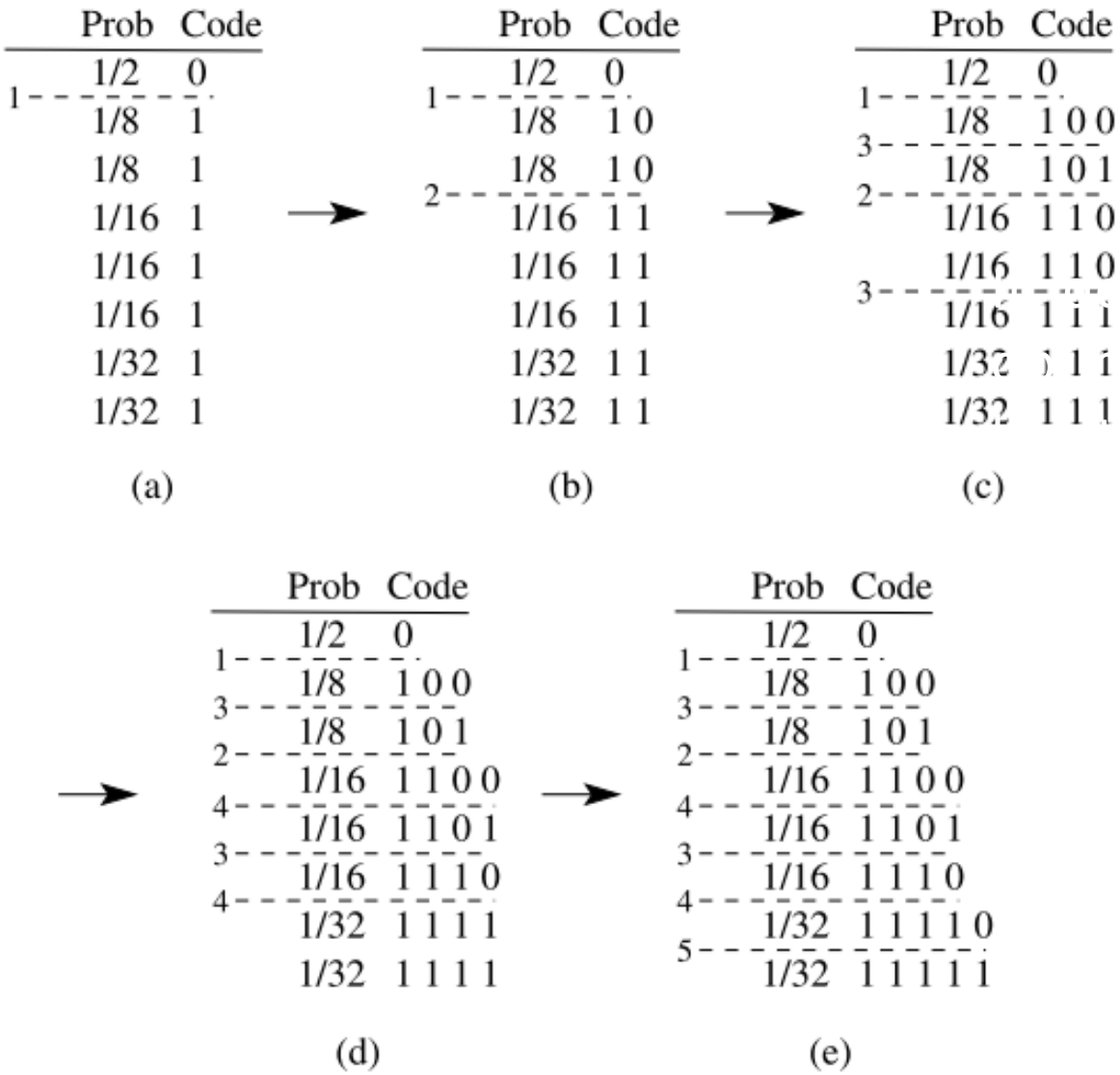
Shannon Fano Coding

Shannon Fano Coding is a top-down binary tree method.

Algorithm:

1. The set of source symbols are sorted in the order of non-increasing probabilities.
2. Source symbol is interpreted as the root of a tree.
3. The list is divided into two groups such that each group has nearly equal total probabilities.
4. The code word of the first group is appended with 0.
5. The code word of the second group is appended with 1.
6. Steps 3-5 are repeated for each group until each group contains only one symbol.

Shannon Fano Coding



Huffman Coding

[David A. Huffman, 1952]

Huffman Coding

The Huffman code is a minimum length code.

This means that given the statistical distribution of the gray levels (the histogram), the Huffman algorithm will generate a code that is as close as possible to the minimum bound, the entropy.

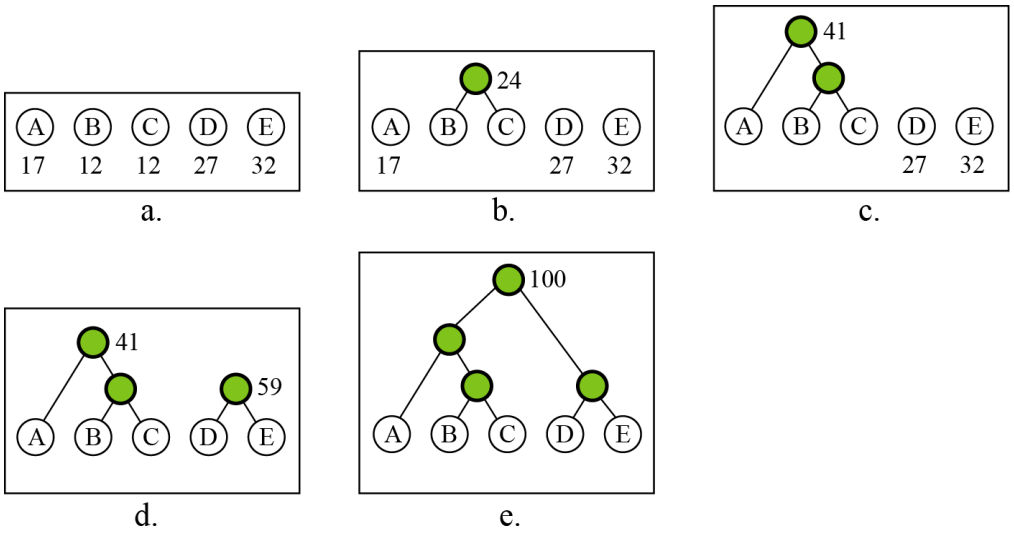
The method results in an unequal (or variable) length code, where the size of the codewords can vary.

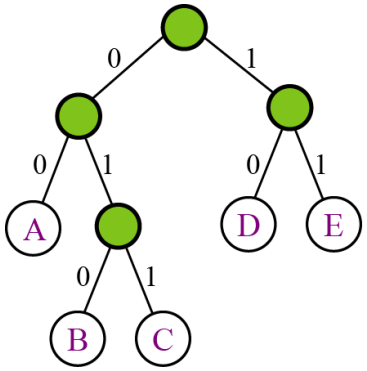
For complex images, Huffman coding alone will typically reduce the file by 10% to 50% (1.1:1 to 1.5:1), but this ratio can be improved to 2:1 or 3:1 by preprocessing for irrelevant information removal.

The Binary Huffman algorithm

1. Find the gray level probabilities for the image by finding the histogram.
2. Order the input probabilities (histogram magnitudes) from smallest to largest.
3. Combine the smallest two by addition. We reorder the probabilities, if necessary.
4. Repeats Step 2-3, until only two probabilities are left.
5. By working backward along the tree, generate code by alternating assignment of 0 and 1.

Binary Huffman Coding



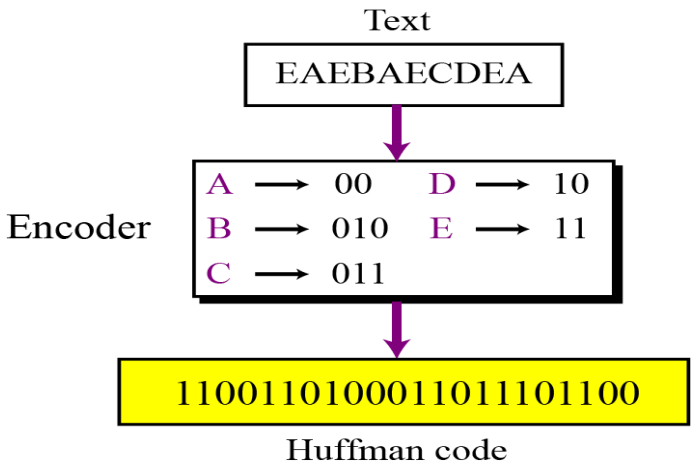


A: 00	D: 10
B: 010	E: 11
C: 011	

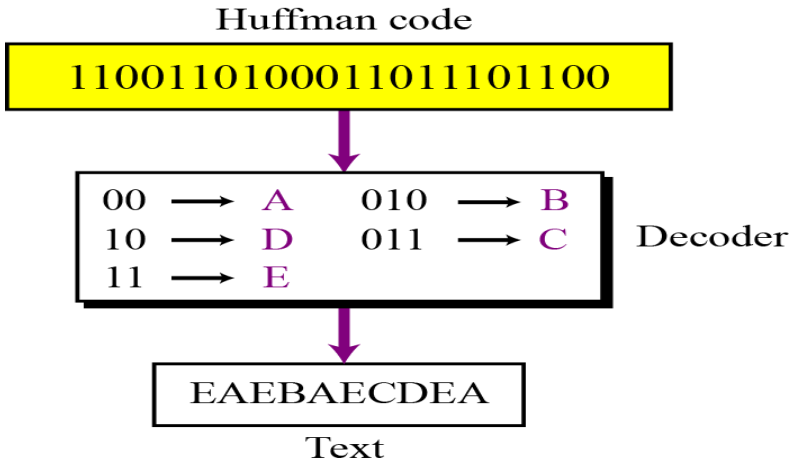
Code

Binary Huffman Coding

Encoding



Decoding



EXAMPLE 10.2.3:

We have an image with 2-bits per pixel, giving four possible gray levels. The image is 10 rows by 10 columns.

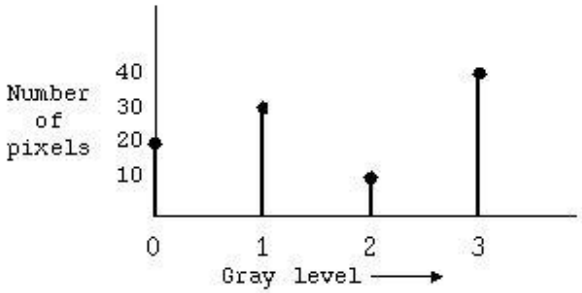
In Step 1 we find the histogram for the image. This is shown in Figure 10.2-2a, where we see that gray level 0 has 20 pixels, gray level 1 has 30 pixels, gray level 2 has 10 pixels, and gray level 3 has 40 pixels with the value. These are converted into probabilities by normalizing to the total number of pixels in the image.

Next, in step 2, the probabilities are ordered as in Figure 10.2-2b.

For step 3, we combine the smallest two by addition.

Step 4 repeats steps 2 and 3, where we reorder (if necessary) and add the two smallest probabilities as in Figure 10.2-2d. This step is repeated until only two values remain.

Figure 10.2-2: Huffman Coding Example



$g_0 = \frac{20}{100} = 0.2$

$g_1 = \frac{30}{100} = 0.3$

$g_2 = \frac{10}{100} = 0.1$

$g_3 = \frac{40}{100} = 0.4$

a) Step 1: Histogram

$g_3 \rightarrow 0.4$

$g_1 \rightarrow 0.3$

$g_0 \rightarrow 0.2$

$g_2 \rightarrow 0.1$

b) Step 2: Order

$0.4 \rightarrow 0.4$

$0.3 \rightarrow 0.3$

$0.2 \rightarrow 0.3$

0.1

c) Step 3: Add

$0.4 \rightarrow 0.4 \rightarrow 0.4$

$0.3 \rightarrow 0.3 \rightarrow 0.6$

$0.2 \rightarrow 0.3$

0.1

$0.4 \rightarrow 0.4 \rightarrow 0.6$

$0.3 \rightarrow 0.3 \rightarrow 0.4$

$0.2 \rightarrow 0.3$

0.1

d) Step 4: Reorder and add until only two values remain

EXAMPLE 10.2.3 (Contd):

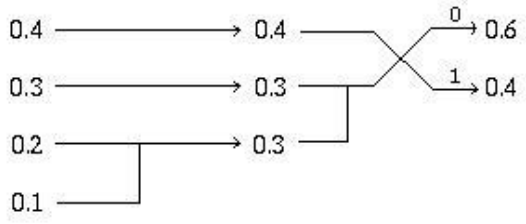
Since we have only two left in our example, we can continue to step 5 where the actual code assignment is made. The code assignment is shown in Figure 10.2-3. We start on the right-hand side of this tree and assign 0's and 1's, working our way back to the original probabilities.

Figure 10.2-3a shows the first assignment of 0 and 1. A 0 is assigned to the 0.6 branch, and a 1 to the 0.4 branch. In Figure 10.2-3b, the assigned 0 and 1 are brought back along the tree, and wherever a branch occurs the code is put on both branches.

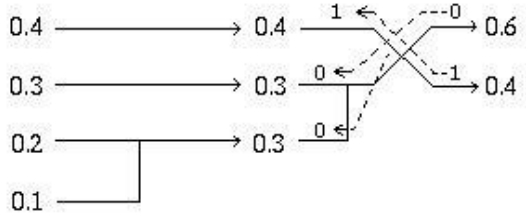
Now (Figure 10.2-3c), we assign the 0 and 1 to the branches labeled 0.3, appending to the existing code. Finally (Figure 10.2-3d), the codes are brought back one more level, and where the branch splits another assignment of 0 and 1 occurs (at the 0.1 and 0.2 branch).

Now we have the Huffman code for this image in Table 10-1.

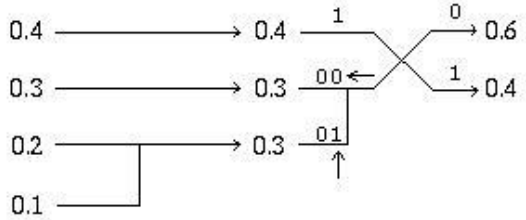
Figure 10.2-3: Huffman Coding Example, Step 5



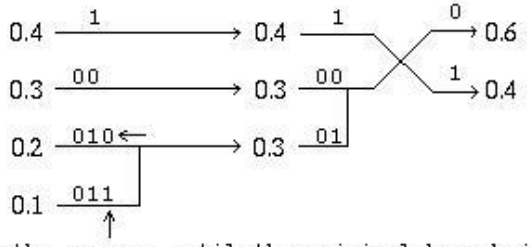
a) Assign 0 and 1 to the right-most probabilities



b) Bring 0 and 1 back along the tree



c) Append 0 and 1 to previously-added branches



d) Repeat the process until the original branch is labeled

TABLE 10-1

Original Gray Level (Natural Code)	**Probability**	**Huffman code**
g_0: 00_2	0.2	010_2
g_1: 01_2	0.3	00_2
g_2: 10_2	0.1	011_2
g_3: 11_2	0.4	1_2

EXAMPLE 10.2.4:

$$Entropy = -\sum_{i=0}^{3} p_i \log_2(p_i)$$

$$= -[(0.2)\log_2(0.2) + (0.3)\log_2(0.3) + (0.1)\log_2(0.1) + (0.4)\log_2(0.4)$$

$$\approx 1.846 \text{ bits/pixel}$$

(Note : $\log_2(x)$ can be found by taking $\log_{10}(x)$ and multiplying by 3.322)

$$L_{ave} = \sum_{i=0}^{L-1} l_i \, p_i$$

$$= 3(0.2) + 2(0.3) + 3(0.1) + 1(0.4)$$

$$= 1.9 \text{ bits/pixel} \quad \text{(Average length with Huffman code)}$$

$$Huffman\ Code\ Efficiency = \frac{Entropy}{L_{ave}} = \frac{1.846}{1.9} = 0.9716 = 97.16\%$$

- In the example, we observe a 2.0 : 1.9 compression, which is about a 1.05 compression ratio, providing about 5% compression.

- Huffman code is highly dependent on the histogram, so any preprocessing to simplify the histogram will help improve the compression ratio.

Arithmetic Coding

[Proposed by Peter Elias (1960). Later, Jorma Rissanen & Richard Pasco developed its practical implementation (1976).]

Arithmetic Coding

We have just seen Huffman coding scheme assigns a code to each symbol individually so that on an average it approaches optimum coding.

When symbols are coded independently based on the statistics of their occurrence, it may not produce least length code for a data stream altogether.

Arithmetic Coding

Suppose a data stream has two symbols ‘a’ and ‘b’ with probabilities 0.90 and 0.10, respectively.

So, entropy (H) of the data stream is 0.47.

But, the average code length due to Huffman coding is 1, which is more than double the actual requirement.

Arithmetic Coding

Another problem with Huffman coding is its non-adaptive nature.

One has to scan the data first to collect statistics in order to form the codeword table.

Then actual coding of the symbols is done based on this table, which remains unchanged throughout the coding.

Arithmetic Coding

Arithmetic coding transforms input data into a single floating point number between 0 and 1.

There is not a direct correspondence between the code and the individual pixel values.

No need to keep codebook like Huffman code.

Need only statistics of the source.

As the images are very large and the precision of digital computers is finite, the entire image must be divided into small subimages to be encoded.

Arithmetic coding uses the probability distribution of the data (histogram), so it can theoretically achieve the maximum compression specified by the entropy.

It works by successively subdividing the interval between 0 and 1, based on the placement of the current pixel value in the probability distribution.

Arithmetic Coding (Example)

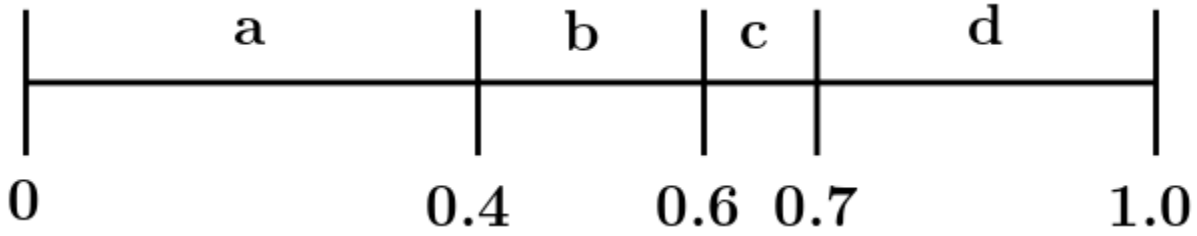
Prob: A source emits four symbols {a,b,c,d} with probabilities 0.4, 0.2, 0.1, and 0.3 respectively. Construct arithmetic coding to encode and decode the word “dad”.

Soln:

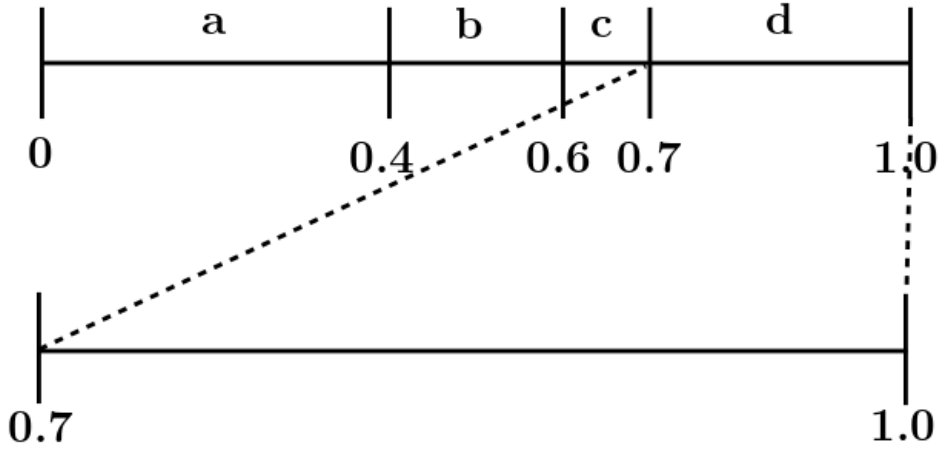
Symbol	a	b	c	d
Probability	0.4	0.2	0.1	0.3
Sub-range	(0.0-0.4)	(0.4-0.6)	(0.6-0.7)	(0.7-1.0)

Arithmetic Coding (Encoder)

Step 1:



Step 2a: The first symbol to be transmitted is d. Hence, the new range is from 0.7 to 1.0



Arithmetic Coding (Encoder)

Step 2b: Find the sub-range for each symbol in between the intervals 0.7 to 1.0.

$$low = low + range \times (low_range)$$
$$high = low + range \times (high_range)$$

i. Interval for symbol 'a'

$$low = 0.7 + (1.0 - 0.7) \times 0 = 0.7$$
$$high = 0.7 + (1.0 - 0.7) \times 0.4 = 0.82$$

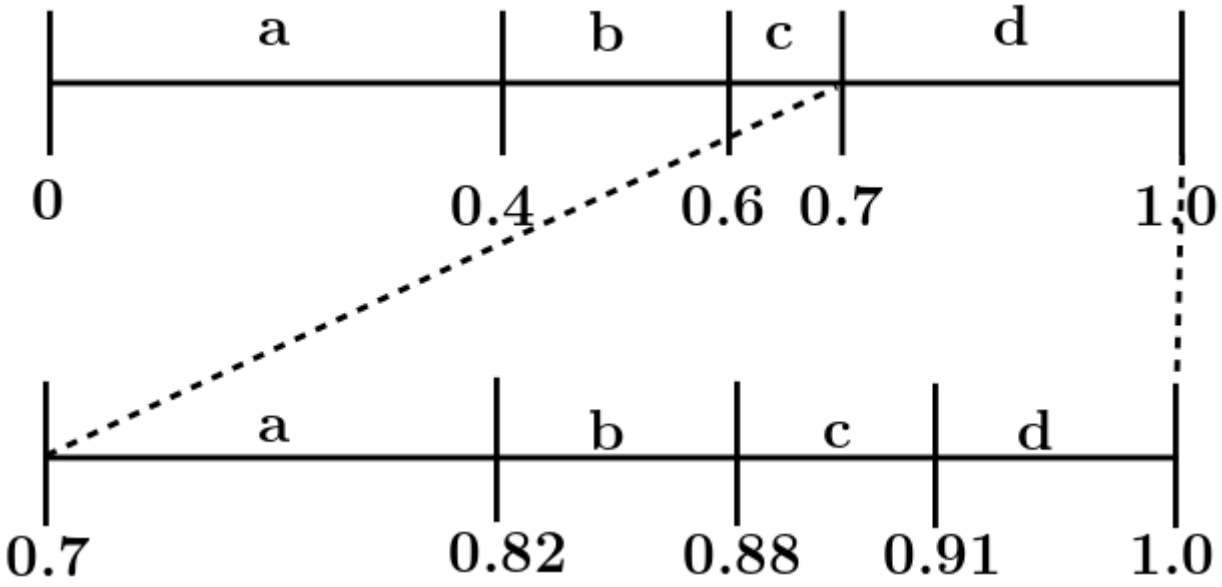
ii. Interval for symbol 'b'

$$low = 0.7 + (1.0 - 0.7) \times 0.4 = 0.82$$
$$high = 0.7 + (1.0 - 0.7) \times 0.6 = 0.88$$

iii. Interval for symbol 'c'

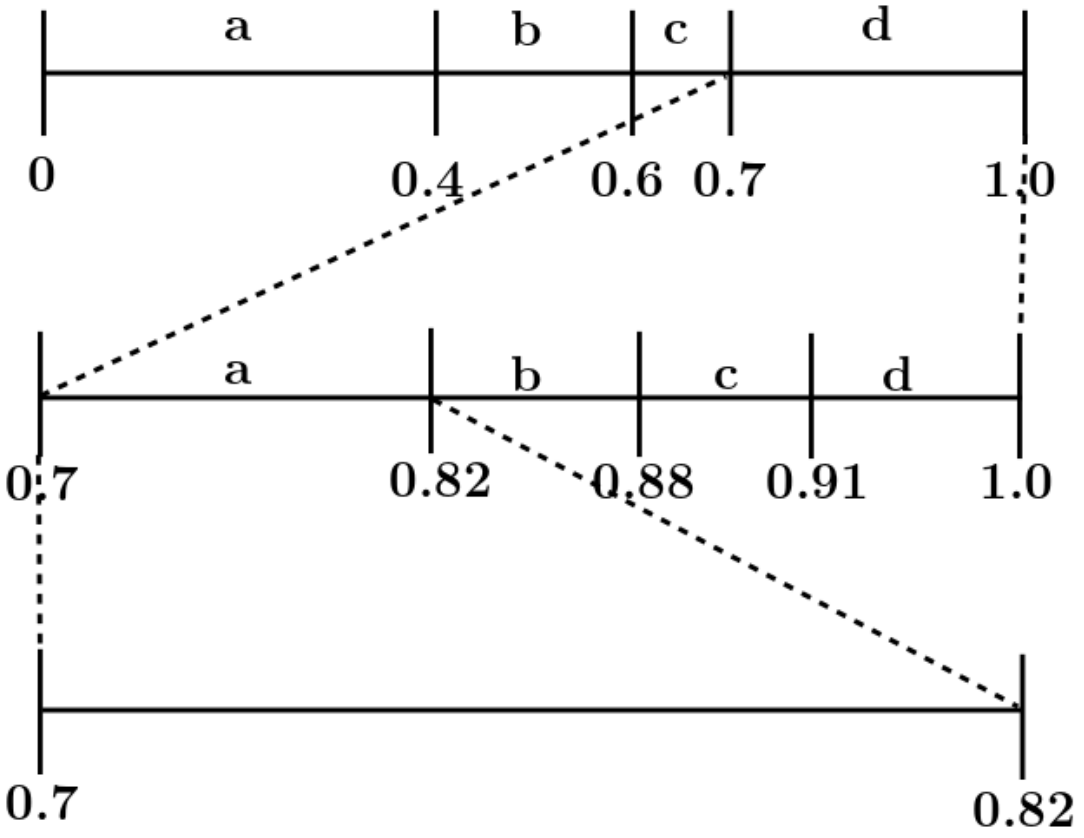
$$low = 0.7 + (1.0 - 0.7) \times 0.6 = 0.88$$
$$high = 0.7 + (1.0 - 0.7) \times 0.7 = 0.91$$

Arithmetic Coding (Encoder)



Arithmetic Coding (Encoder)

Step 3: The symbol to be transmitted is ‘a’.



Arithmetic Coding (Encoder)

Step 3a: Interval for ‘a’.

$$low = 0.7 + (0.82 - 0.7) \times 0.0 = 0.7$$
$$high = 0.7 + (0.82 - 0.7) \times 0.4 = 0.748$$

Step 3b: Interval for ‘b’.

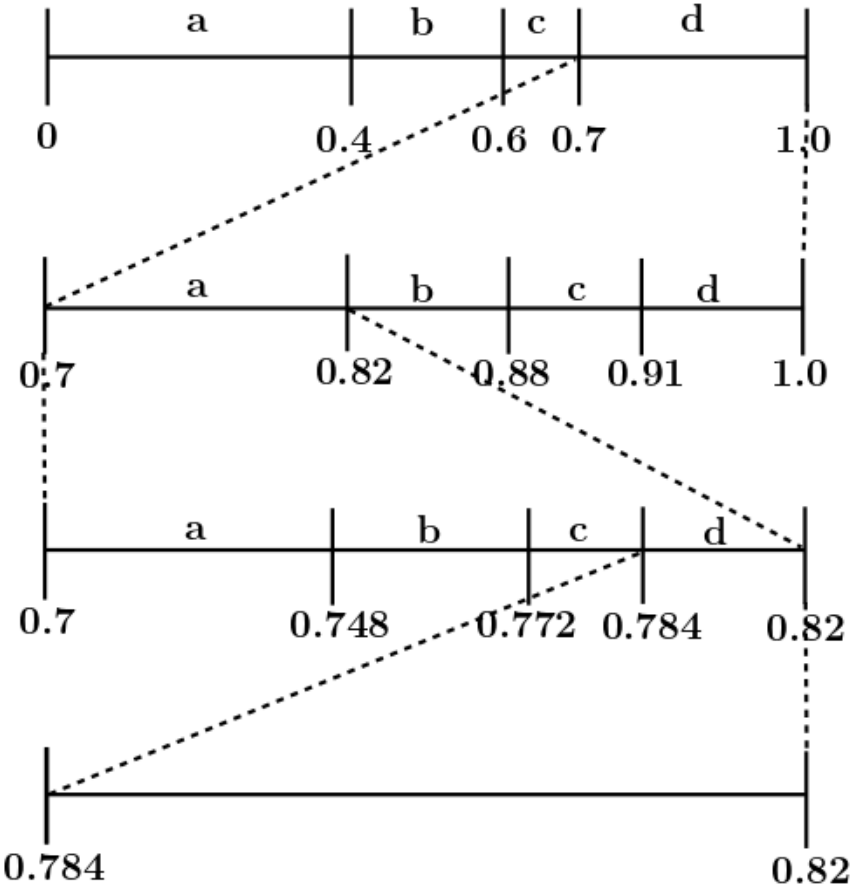
$$low = 0.748 \; and \; high = 0.772$$

Step 3c: Interval for ‘c’.

$$low = 0.772 \; and \; high = 0.784$$

Arithmetic Coding (Encoder)

Step 4: The symbol to be transmitted is ‘d’.

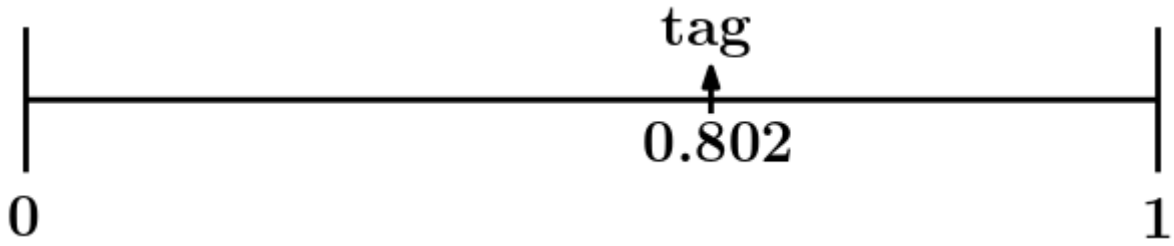


Arithmetic Coding (Encoder)

Step 5: tag value = $\frac{0.784+0.82}{2}$ = 0.802

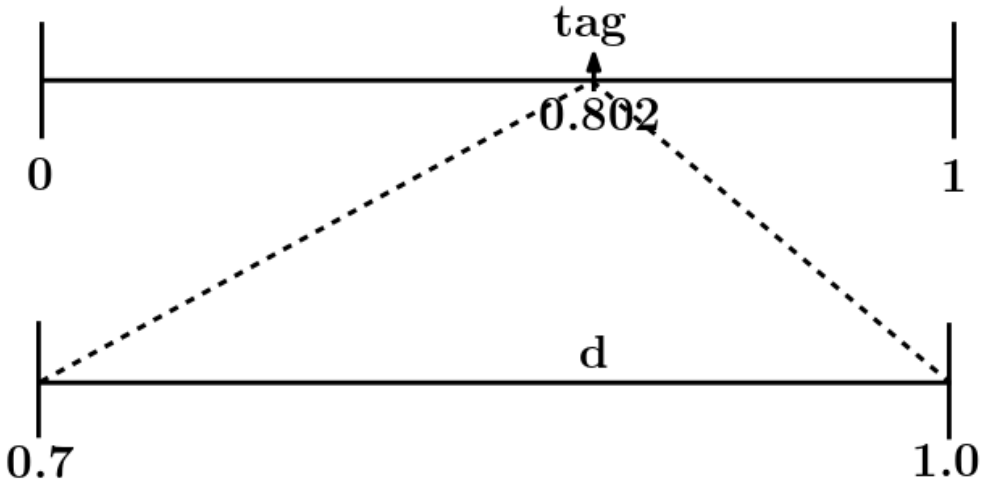
Arithmetic Coding (Decoder)

Step 1: The tag received by the receiver is 0.802



- The tag value is compared with the symbol sub-range. It is in between 0.7 and 1.0. The corresponding decoded symbol is d.

Step 2:

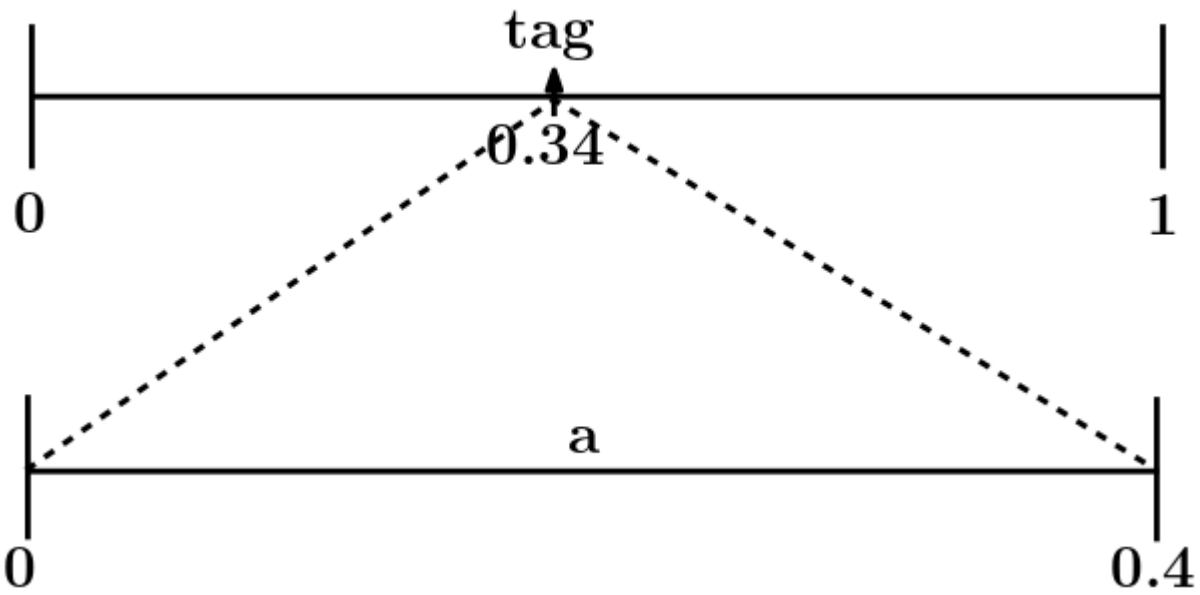


Arithmetic Coding (Decoder)

Step 3: The new tag value-

$$t^* = \frac{tag-low}{range} = \frac{0.802-0.7}{0.3} = 0.34$$

Step 4:

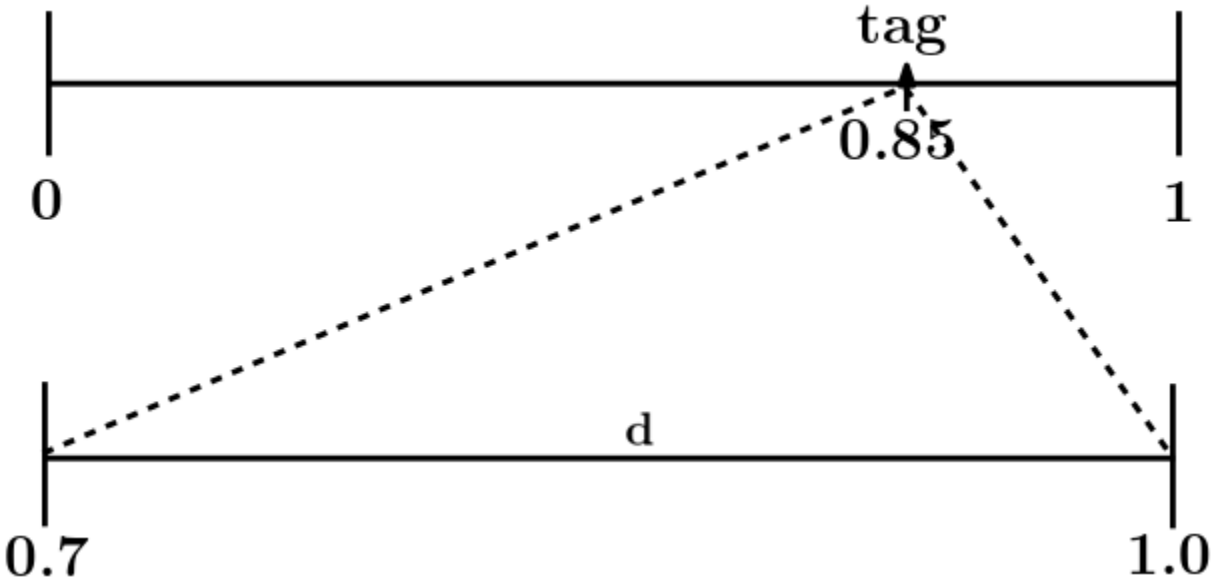


Arithmetic Coding (Decoder)

Step 5: The new tag value-

$$t^* = \frac{tag-low}{range} = \frac{0.34-0}{0.4} = 0.85$$

It is in between 0.7 and 1.0. Decoded symbol is d.



Disadvantage of Arithmetic Coding

The main disadvantage is that it tends to be slow. We have seen that the full precision form of arithmetic coding requires at least one multiplication/division per event and in some implementations up to two multiplications and two divisions per event.

In practice, this technique may be used as part of an image compression scheme, but is impractical to use alone.

It is one of the options available in the JPEG standard.

Lossy Compression Methods

Lossy compression methods are required to achieve high compression ratios with complex images.

They provide adjustments between image quality and degree of compression, which allows the compression algorithm to be customized to the application.

Lossy Image Compression



a) Original image



b) JPEG compression, 10:1 ratio



c) JPEG compression, 48:1 ratio



d) Wavelet/vector quantization compression, 36:1 ratio

With more advanced methods, images can be compressed 10 to 20 times with virtually no visible information loss, and 30 to 50 times with minimal degradation.

Newer techniques, such as JPEG2000, can achieve reasonably good image quality with compression ratios as high as 100 to 200.

Image enhancement and restoration techniques can be combined with lossy compression schemes to improve the appearance of the decompressed image.

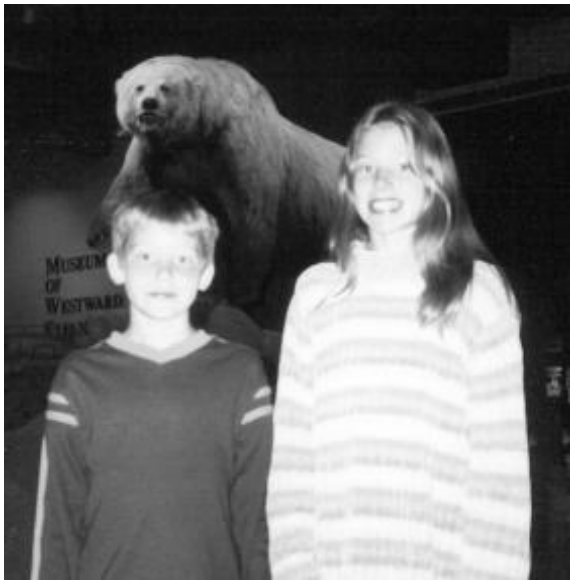
In general, a higher compression ratio results in a poorer image, but the results are highly image dependent – application specific.

Lossy compression can be performed in both the spatial and transform domains. Hybrid methods use both domains.

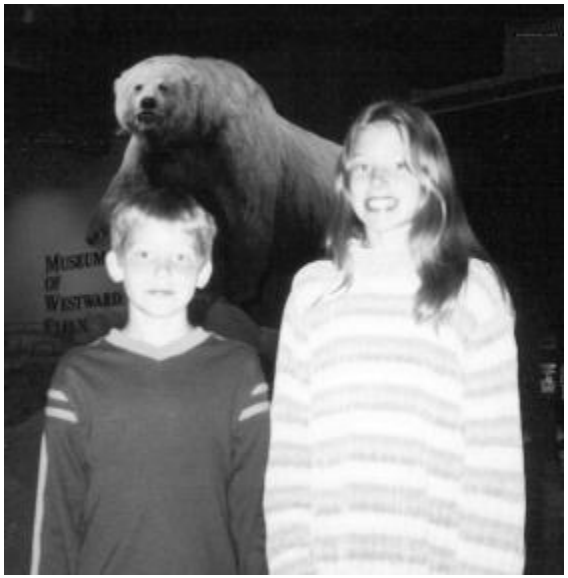
Gray-Level Run Length Coding

The RLC technique can also be used for lossy image compression, by reducing the number of graylevels, and then applying standard RLC techniques.

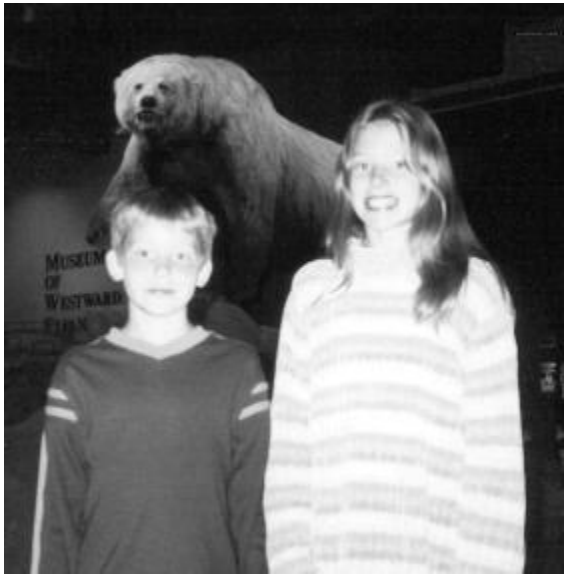
As with the lossless techniques, preprocessing technique will improve the compression ratio.



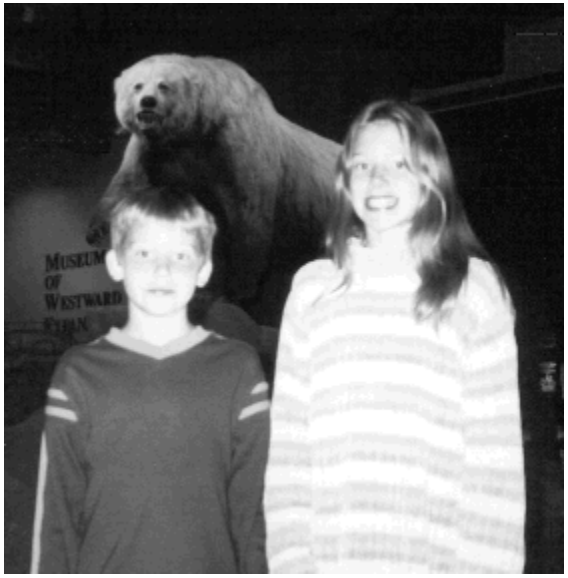
a) Original image, 8 bits/pixel, 256 gray levels



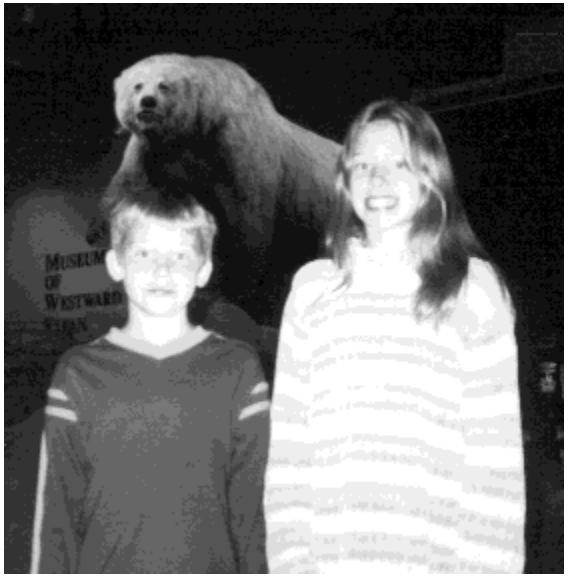
b) Image after reduction to 7 bits/pixel, 128 gray levels, compression ratio 0.55, with preprocessing 0.66



c) Image after reduction to 6 bits/pixel, 64 gray levels, compression ratio 0.77, with preprocessing 0.97



d) Image after reduction to 5 bits/pixel, 32 gray levels, compression ratio 1.20, with preprocessing 1.60



e) Image after reduction to 4 bits/pixel, 16 gray levels, compression ratio 2.17, with preprocessing 2.79



f) Image after reduction to 3 bits/pixel, 8 gray levels, compression ratio 4.86, with preprocessing 5.82



g) Image after reduction to 2 bits/pixel,
4 gray levels, compression ratio 13.18,
with preprocessing 15.44



h) Image after reduction to 1 bit/pixel,
2 gray levels, compression ratio 44.46,
with preprocessing 44.46

Block Truncation Coding

[First presented in a paper by Delp & Mitchell, 1979]

Block Truncation Coding

Block Truncation Coding (BTC) technique is based on preserving the first and second statistical moments of the image.

BTC works by dividing the image into small subimages (blocks) and then reducing the number of graylevels within each block.

The graylevels are reduced by a quantizer that adapts to local statistics.

The levels for the quantizer are chosen to minimize a specified error criteria, and then all the pixel values within each block are mapped to the quantized levels.

The necessary information to decompress the image is then encoded and stored.

The basic form of BTC divides the image into N×N blocks and codes each block using a two-level quantizer.

The two levels are selected so that the mean and variance of the gray levels within the block are preserved.

Each pixel value within the block is then compared with a threshold, typically the block mean, and then is assigned to one of the two levels.

If it is above the mean it is assigned the high level code, if it is below the mean, it is assigned the low level code.

If we call the high value *H* and the low value *L*, we can find these values via the following equations:

$$H = m_b + \sigma_b \sqrt{\frac{n^2 - q}{q}}$$

$$L = m_b - \sigma_b \sqrt{\frac{q}{n^2 - q}}$$

Where the block size is $n \times n$

$b =$ *the current block*

$m_b =$ *the block mean* $= \frac{1}{n^2} \sum_{I(r,c) \in b} I(r,c)$

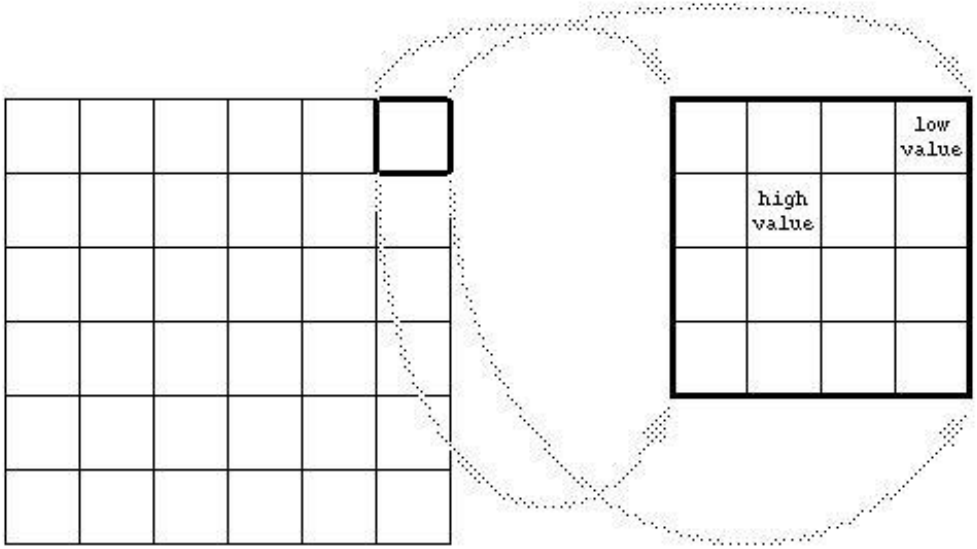
$\sigma_b =$ *the block standard deviaiton* $= \sqrt{\frac{1}{n^2} \sum_{I(r,c) \in b} [I(r,c)]^2 - m_b^2}$

$q =$ *the number of values in the block* $\geq m_b$

If $n = 4$, then after the H and L values are found, the 4x4 block is encoded with four bytes.

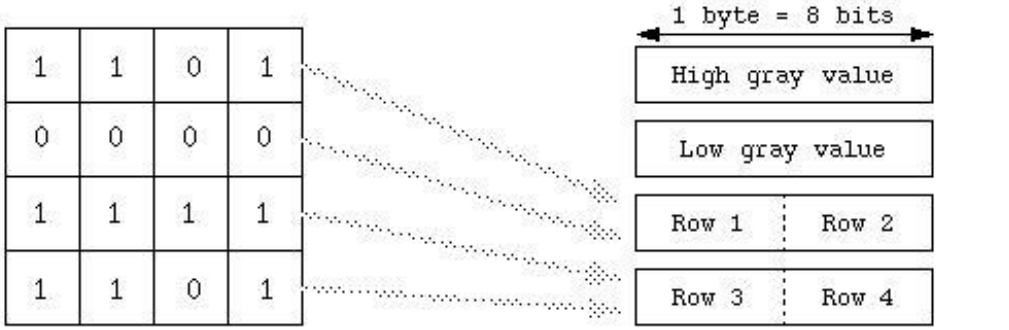
Two bytes to store the two levels, H and L, and two bytes to store a bit string of 1's and 0's corresponding to the high and low codes for that particular block.

Figure 10.3-4: Basic Block Truncation Coding



a) Divide image into 4x4 blocks

b) Find high and low values for blocks



c) Assign a '0' to each pixel less than the mean, '1' to each pixel greater than the mean

d) Encode 4x4 block with 4 bytes

EXAMPLE 10.3.2:

Given the following 4x4 subimage, apply basic BTC and find the resulting values.

$$\begin{bmatrix} 12 & 16 & 15 & 17 \\ 13 & 16 & 17 & 17 \\ 4 & 4 & 35 & 35 \\ 42 & 42 & 12 & 12 \end{bmatrix}$$

$$m_b = \frac{1}{n^2} \sum_{I(r,c) \in b} I(r,c) = \frac{1}{16}\left[12+16+15+17+13+16+17+17+4+4+35+35+42+42+12+12\right]$$

$$= 19.3125$$

$$\sigma_b = \sqrt{\frac{1}{n^2} \sum_{I(r,c) \in b} [I(r,c)]^2 - m_b^2}$$

$$= \sqrt{\frac{1}{16}\left[12^2+16^2+15^2+17^2+13^2+16^2+17^2+17^2+4^2+4^2+35^2+35^2+42^2+42^2+12^2+12^2\right] - (19.3125)^2}$$

$$\approx 11.85$$

EXAMPLE 10.3.2 (contd):

There are 4 pixels values greater than the mean, so q = 4.

$$H = m_b + \sigma_b \sqrt{\frac{n^2 - q}{q}} = 19.3125 + 11.85 \sqrt{\frac{16-4}{4}} \approx 40$$

$$L = m_b - \sigma_b \sqrt{\frac{q}{n^2 - q}} = 19.3125 - 11.85 \sqrt{\frac{4}{16-4}} \approx 13$$

Now, find the bit string by using 0 for values less than the mean and 1 for values greater than the mean:

$$\begin{bmatrix} 12 & 16 & 15 & 17 \\ 13 & 16 & 17 & 17 \\ 4 & 4 & 35 & 35 \\ 42 & 42 & 12 & 12 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \Rightarrow (0000000000111100_2)$$

The high value, H, and the low value, L, will be stored along with the bit string. The subimage, when decompressed will be:

$$\begin{bmatrix} 13 & 13 & 13 & 13 \\ 13 & 13 & 13 & 13 \\ 13 & 13 & 40 & 40 \\ 40 & 40 & 13 & 13 \end{bmatrix}$$

This algorithm tends to produce images with blocky effects.

These artifacts can be smoothed by applying enhancement techniques such as median and average (lowpass) filters.

BTC lends itself very nicely to parallel processsing since the coding of each of the blocks is totally independent.

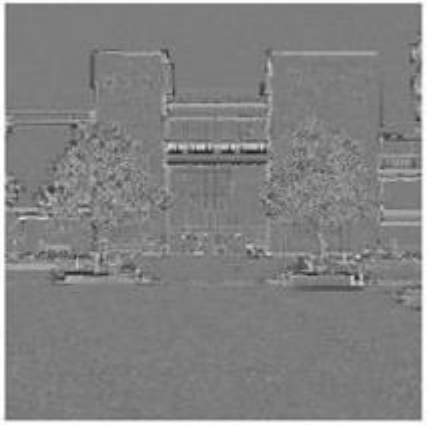
Block Truncation Coding (BTC)



a) Original image



b) Block truncation coded image, compression = 4:1



c) Error image of (b), histogram stretched to show detail

continued



d) Image (b) post-processed with a 3x3 median filter



e) Image (b) post-processed with a 3x3 averaging filter

The multilevel BTC algorithm, which uses a 4-level quantizer, allows for varying the block size, and a larger block size should provide higher compression, but with a corresponding decrease in image quality.

Multilevel Block Truncation Coding



a) Original image



b) Multilevel BTC,
block size = 8x8



c) Error image of (b),
multiplied to show detail

continued



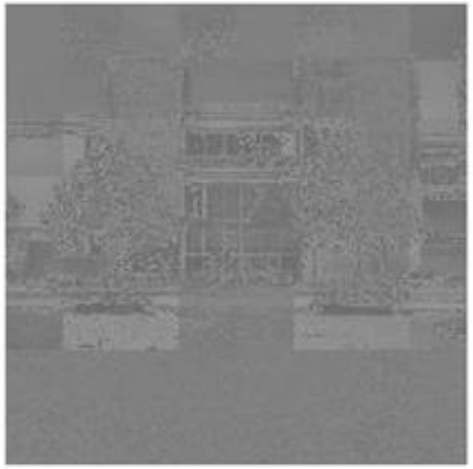
d) Multilevel BTC,
block size = 16x16,



e) Error image of (d)
multiplied to show detail



f) Multilevel BTC,
block size = 32x32



g) Error image of (f)
multiplied to show detail

Vector Quantization

[Primarily developed by Robert M. Gray, 1980]

Vector Quantization

Vector quantization (VQ) is the process of mapping a vector that can have many values to a vector that has a smaller (quantized) number of values.

For image compression, the vector corresponds to a small subimage or block.

EXAMPLE 10.3.3:

Given the following 4x4 subimage:

$$\begin{bmatrix} 65 & 70 & 71 & 75 \\ 71 & 70 & 71 & 81 \\ 81 & 80 & 81 & 82 \\ 90 & 90 & 91 & 92 \end{bmatrix}$$

This can be re-arranged into a 1-D vector by putting the rows adjacent as follows:

[row1 row2 row3 row4] = [65 70 71 75 71 70 71 81 81 80 81 82 90 90 91 92]

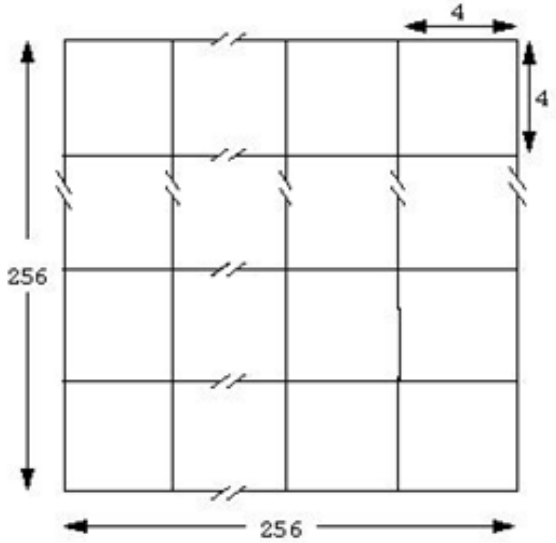
VQ can be applied in both the spectral or spatial domains.

Information theory tells us that better compression can be achieved with vector quantization than with scalar quantization (rounding or truncating individual values).

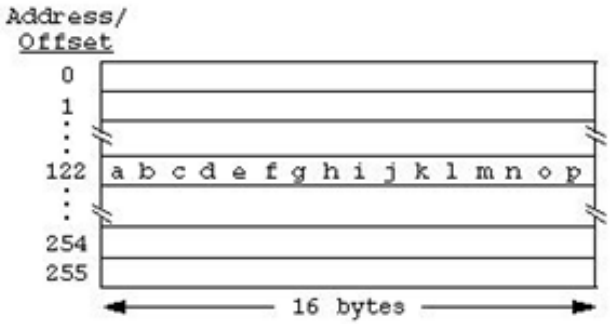
Vector quantization treats the entire subimage (vector) as a single entity and quantizes it by reducing the total number of bits required to represent the subimage.

This is done by utilizing a codebook, which stores a fixed set of vectors, and then coding the subimage by using the index (address) into the codebook.

Example



a) Original 256x256 image divided into 4x4 blocks



b) Codebook with 256 16-byte entries

a	b	c	d
e	f	g	h
i	j	k	l
m	n	o	p

c) A subimage decompressed with vector #122

Given an 8-bit 256×256 image, we device a vector quantization scheme that will encode each 4×4 block with one of the vectors in a codebook of 256 entries.

We determine that we want to encode a specific subimage with vector number 122 in the codebook. For this subimage we then store the number 122 as the index into the codebook.

Then when the image is decompressed, the vector at the 122 address of the codebook, is used for that particular subimage.

Compression ratio is 16:1

In the example we achieved a 16:1 compression, but note that this assumes that the codebook is not stored with the compressed file.

Codebook needs to be stored unless a generic codebook is devised which could be used for a particular type of image, in that case we need only store the name of that particular codebook file.

In the general case, better results will be obtained with a codebook that is designed for a particular image.

EXAMPLE 10.3.5:

If we include the codebook in the compressed file from the previous example, the compression ratio will not be quite as good. For every 4x4 block we will have 1 byte. This gives us:

$$\left(\frac{256\ pixels}{4\ pixels/block}\right)\left(\frac{256\ pixels}{4\ pixels/block}\right)=4096\ blocks$$

At 1 byte for each 4x4 block, this give us 4096 bytes for the codebook addresses. Now we also include the size of the codebook, 256x16:

$$4096+(256)(16)=8192\ bytes\ for\ the\ coded\ file$$

The original 8-bit, 256x256 image contained:

$$(256)(256)=65,536\ bytes$$

Thus, we obtain a compression of:

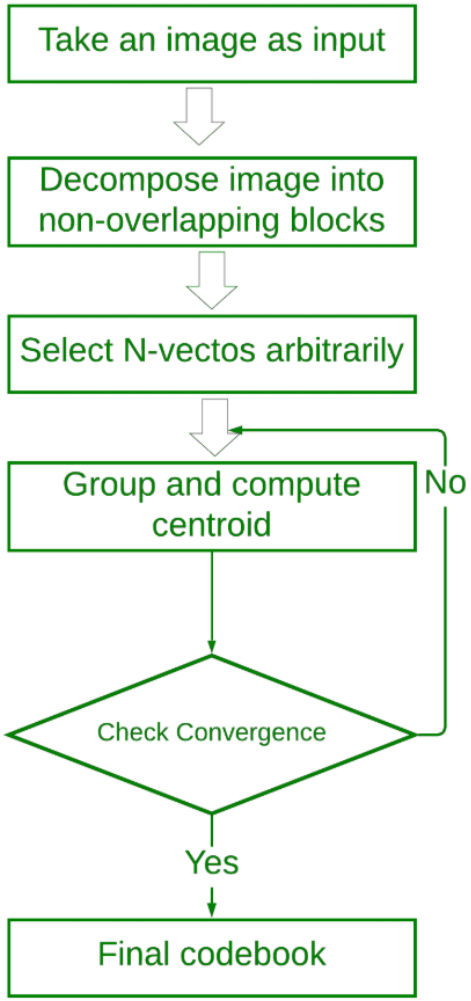
$$\frac{65,536}{8192}=8\rightarrow 8:1\ compression$$

In this case, including the codebook cut the compression in half, from 16:1 to 8:1

A training algorithm determines which vectors will be stored in the codebook by finding a set of vectors that best represent the blocks in the image.

This set of vectors is determined by optimizing some error criterion, where the error is defined as the sum of the vector distances between the original subimages and the resulting decompressed subimages.

The standard algorithm to generate the codebook is the Linde-Buzo-Gray (*LBG*) algorithm, also called the *K-means* or the *clustering* algorithm:



The LBG algorithm, along with other iterative codebook design algorithms do not, in general, yield globally optimum codes.

These algorithms will converge to a local minimum in the error (distortion) space.

Theoretically, to improve the codebook, the algorithm is repeated with different initial random codebooks and the one codebook that minimizes distortion is chosen.

The primary advantage of vector quantization is simple and fast decompression, but with the high cost of complex compression.

The decompression process requires the use of the codebook to recreate the image, which can be easily implemented with a look-up table (LUT).

This type of compression is useful for applications where the images are compressed once and decompressed many times, such as images on an Internet site.



a) Original image



b) VQ with 4x4 vectors, and a codebook of 128 entries, compression ratio = 11.49



c) VQ with 4x4 vectors, and a codebook of 256 entries, compression ratio = 7.93



d) VQ with 4x4 vectors, and a codebook of 512 entries, compression ratio = 5.09

Note: As the codebook size is increased, the image quality improves and the compression ratio decreases.

Transform Coding

Transform Coding

Transform coding, is a form of block coding done in the transform domain.

The image is divided into blocks, or subimages, and the transform is calculated for each block.

Any of the defined transforms can be used, e.g. Fourier or Walsh-Hadamard, but it has been determined that the discrete cosine transform (DCT) is optimal for most images.

The JPEG standard uses DCT and the newer version of this, i.e., JPEG2000 algorithm uses the wavelet transform, which has been found to provide even better compression.

This type of transform is a special type of mapping that uses spatial-frequency concepts as a basis for the mapping.

The main reason for mapping the original data into another mathematical space is to pack the information (or energy) into as few coefficients as possible.



a) VQ with the discrete cosine transform, compression ratio = 9.21



b) VQ with the wavelet transform, compression ratio = 9.21